

Computer Vision:

Homework-6

Bilal Ahmed
ahmedb@purdue.edu

1 Objective

In this assignment, the objective is to implement image segmentation using Otsu algorithm, threshold selection for single channel histograms. Specifically, Otsu algorithm will be applied to RGB channels and texture maps. These maps will then be used for segmentation of foreground. Further implementation details will be mentioned in Algorithm and Results sections.

2 Theoretical Questions

Explain strengths and weaknesses of Watershed and Otsu Algorithm.

Answer: Otsu algorithm is basically a threshold selection algorithm that uses histograms to segment foreground from the background. These thresholds can be applied to RGB channels separately or on multiple texture maps to get masks. These different masks are then combined to get the segmentation of foreground. Choice of using RGB channels or texture maps depends on the foreground object and background. Whereas Watershed algorithm uses gradient map of the image and applies different morphological operations on that. Conceptually, it considers gradient maps as topographical landscape and strives to find the ridges in that landscape. These ridges specify the contours in the image.

Strength of Otsu algorithm is that it can use texture differences in the foreground and background to segregate them and is computationally efficient, but the weakness is that it is mostly suitable segmentation of one foreground object or a few foreground objects from the background (using multiple thresholds). On the other hand, watershed algorithm is computationally costly, but is able to segment image with arbitrary detail.

3 Algorithms & Results

Image Segmentation Using RGB Values

Since Otsu algorithm works on gray-scale (single-channel) images, we can use on RGB channels as follows;

- Apply otsu algorithm separately on RGB channels and get their corresponding maps.
- Combine these RGB masks using logical 'AND' operation.
- Combined mask gives the map of foreground vs background.
- Using contour extraction algorithm (explain later) to get contour of segmentation

Fig.1 and Fig.4 display separate channel masks as well combined masks for task-1 and task-2 respectively. Table. 1 displays the hyper-parameters used for RGB masks and contour extraction.

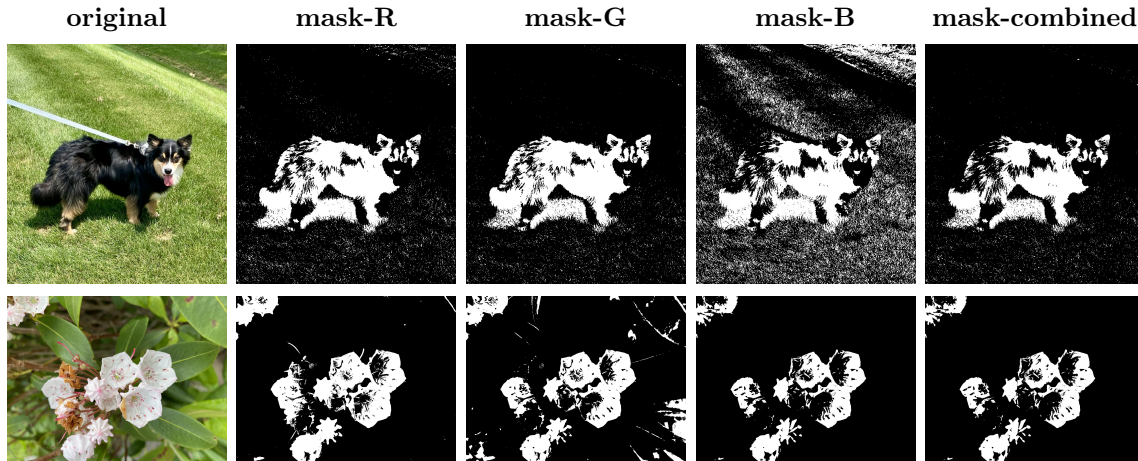


Figure 1: Original images, and segmentation masks using Otsu on RGB channels

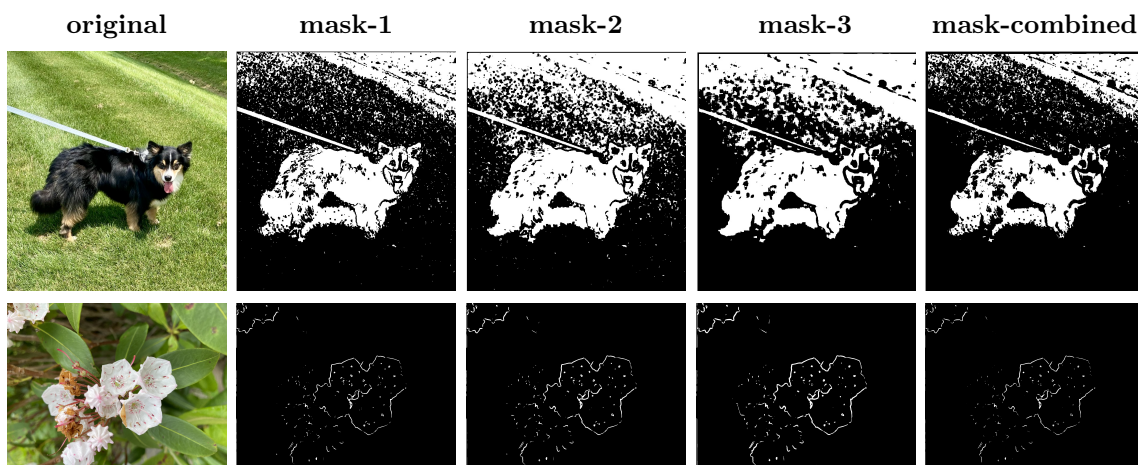


Figure 2: Original images, and segmentation masks using Otsu on texture maps

Table 1: Tuned parameters for segmentation based on RGB channels

image	number of iterations	flip	morph. operation	morph. kernel
flower	[2,2,2]	[False, False, False]	closing	3
dog	[2,2,2]	[True, True, True]	opening	3
fire-hydrant	[2,2,1]	[False, False, False]	opening	3
flowers	[1,1,1]	[False, True, False]	closing	3

Texture Based Segmentation

For texture based segmenation,

- Texture based feature maps are obtained by computing variance of pixels in the $N \times N$ neighbour of each pixel.
- For boundary pixels, the image is padded with zeros.
- Texture maps using different values of N are obtained. Values of N used are mentioned in the Table 2 as 'texture-kernels'.

- Otsu algorithm is applied to each of these texture maps and resulting masks obtained combined to get the combined mask. Individual masks and their combined result are display in Fig.2 (task-1) and Fig.5 (task-2)

Tuned parameters for texture based segmentation like number of iterations, texture kernels, flip setting are given in Table 2 for both task-1 and task-2.

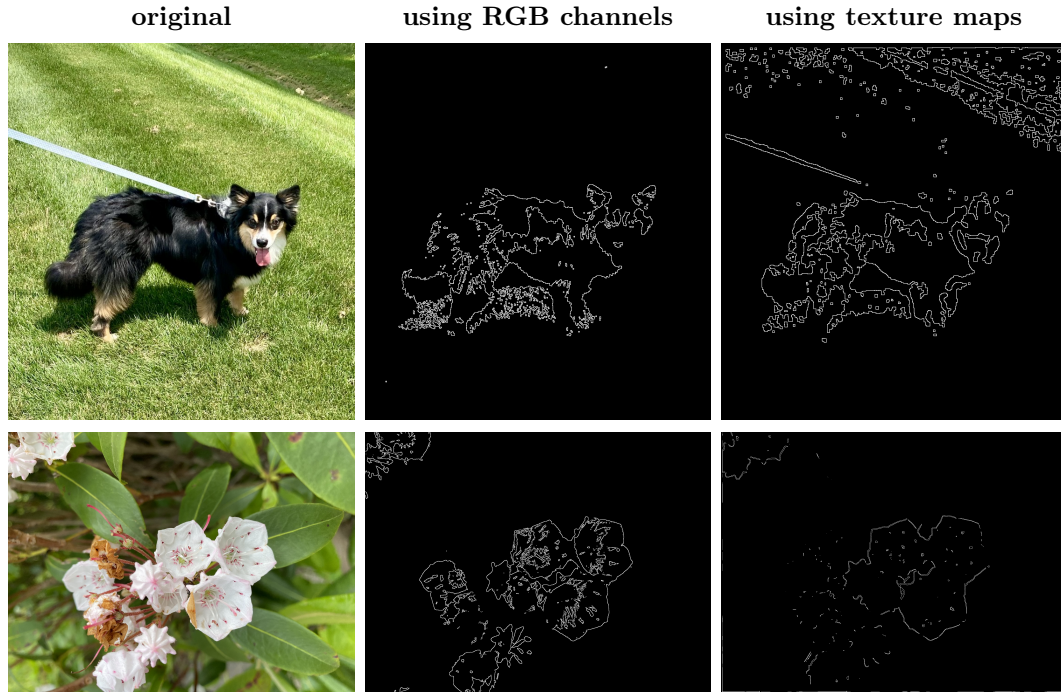


Figure 3: Original images, extracted contours using RGB and texture maps

Table 2: Tuned parameters for segmentation based on texture maps

image	number of iterations	flip	texture kernels	morph. operation	morph. kernel
flower	[1,1,1]	[False, False, False]	[9,11,13]	closing	3
dog	[3,2,2]	[True, True, True]	[5,7, 11]	opening	5
fire-hydrant	[1,1,1]	[False, False, False]	[3,5,7]	morhp. gradient	3
flowers	[1,1,1]	[False, False, False]	[3,5,7]	closing	3

Contour Extraction

- For contour extraction, one of the following three morphological operation is performed on the segmentation mask;
 - **opening**: erosion followed by dilation.
 - **closing**: dilation followed by erosion.
 - **morphological gradient**: dilated image minus eroded image.

The type of morphological operator applied and size of kernel is mentioned in Table 1 and Table 2.

- For each pixel in the mask, that is high (255 or 1) and at least one of its eight neighbors is low (0), set equal to high (255 or 1).

Extracted contours using both RGB channels and texture maps are displayed in Fig.3 and Fig. 6 for task-1 and task-2 respectively.

Results for Task-2

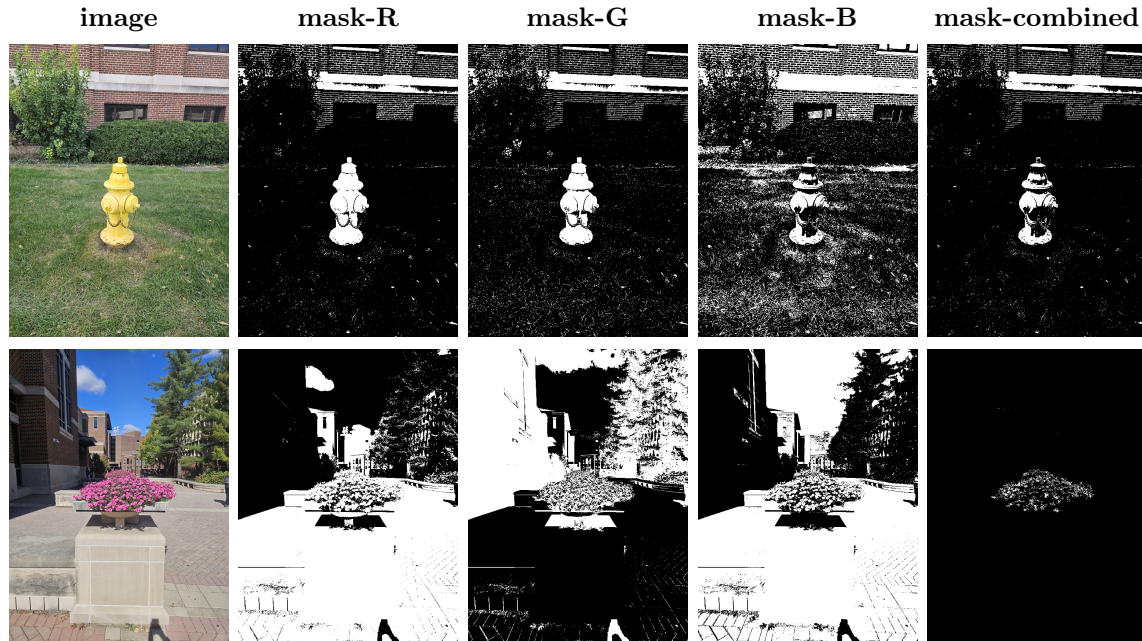


Figure 4: Original images, and segmentation masks using Otsu on RGB channels

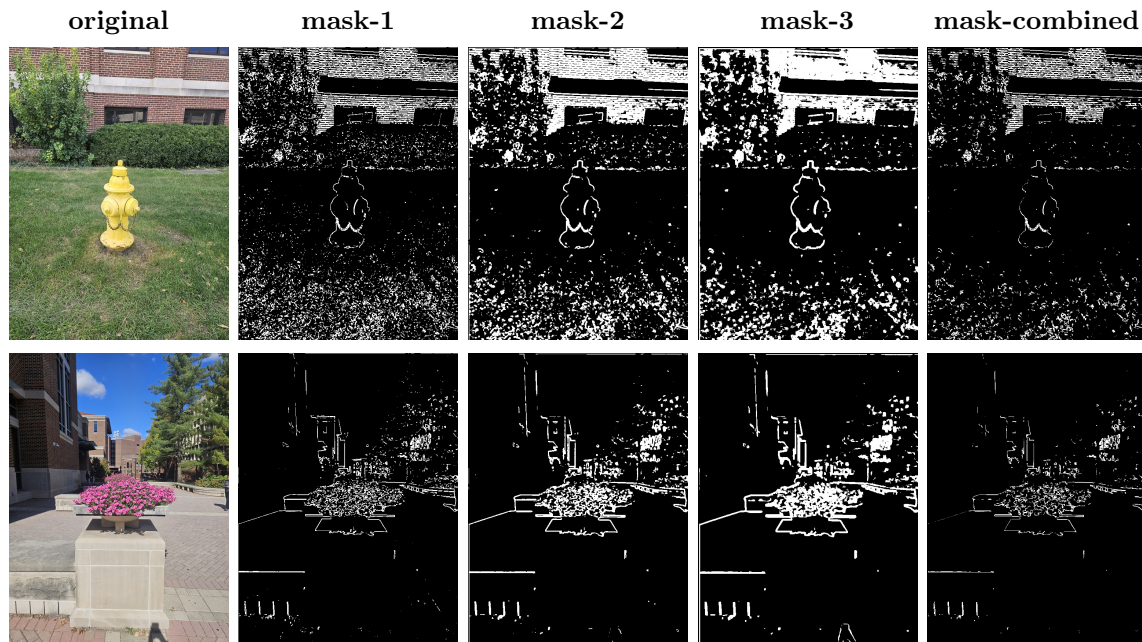


Figure 5: Original images, and segmentation masks using Otsu on texture maps

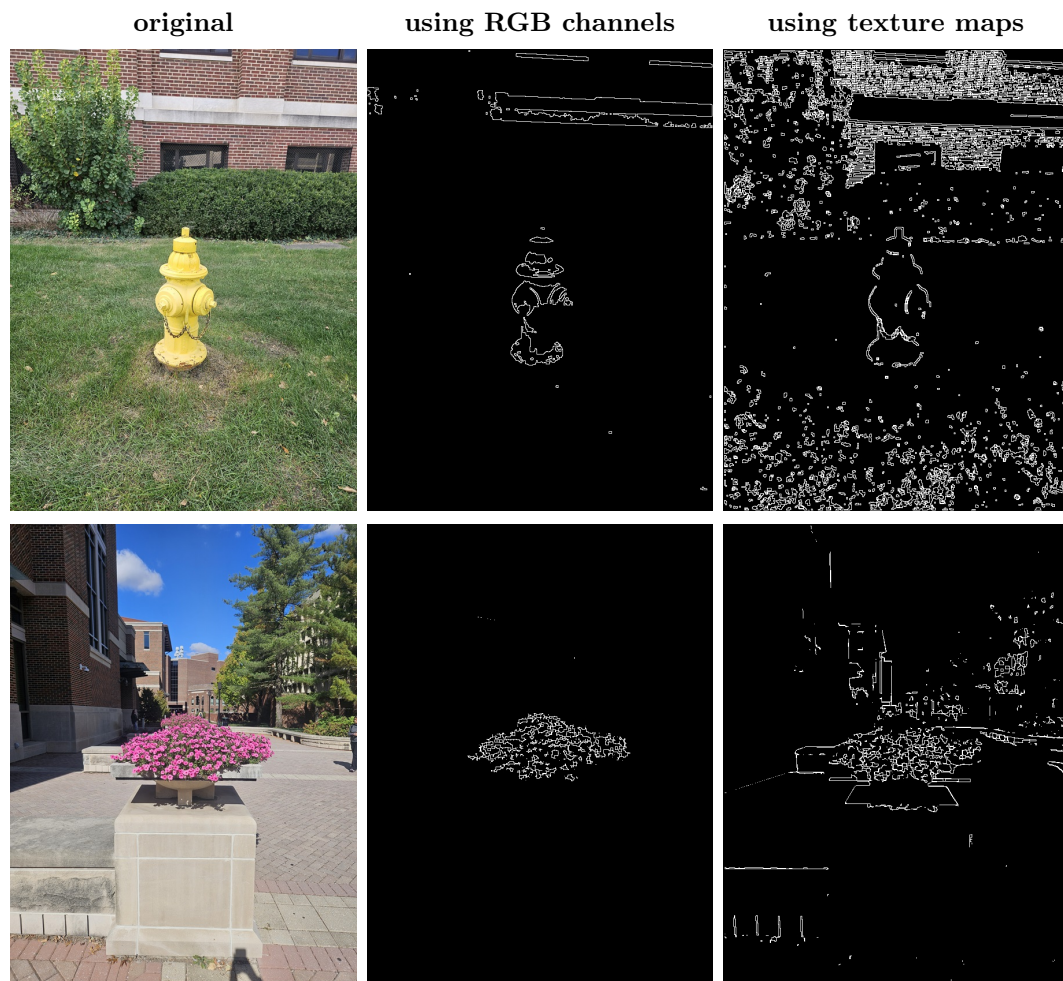


Figure 6: Original images, extracted contours using RGB and texture maps

4 Discussion

Otsu algorithm is flexible and can be applied to RGB channels or texture maps. The choice of approach to take depends on the foreground and background in the image. If the foreground differs from background in terms of colors, using RGB channels would be more useful. In other cases, when the texture of foreground object is very different from texture in the background, using texture maps is the more effective.

For example, the image of flowers has difference of color that appears easy to segregate in the foreground and background, and we can see that results for RGB channel is much better than the texture.

Other things to consider are the level details in the segmentation mask while applying morphological operator for contour extraction. For extracting greater detail, smaller kernel sizes should be used.

Python Code

```

1 import cv2
2 import pathlib
3 # import skimage
4 import scipy
5 import computer_vision

```

```

6
7 import matplotlib.pyplot as plt
8 import matplotlib as mpl
9 import numpy as np
10 import math
11
12
13 def read_image(subdir, image_name):
14     """Reads images from the subdir"""
15     path = pathlib.Path(computer_vision.__path__[0])
16     image_path = path.parents[0] / 'images' / subdir / image_name
17     img_bgr = cv2.imread(image_path)
18     # Convert the image from BGR to RGB format
19     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
20     return img_rgb
21
22 def save_image(image, subdir, image_name, grayscale=False):
23     """Saves image to the subdir"""
24     path = pathlib.Path(computer_vision.__path__[0])
25     image_path = path.parents[0] / 'images' / subdir / image_name
26     # Save the image with specific quality
27     if not grayscale:
28         image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
29     cv2.imwrite(image_path, image)
30
31
32 def otsu_single_itr(img, L=256, mask=None):
33     """Performs one iteration of otsu algorithm.
34
35     Args:
36         img: single level image
37         L: number of of gray levels
38         mask: mask to focus on image regions
39
40     Returns:
41         threshold
42         gray_level_index
43     """
44     epsilon = 1.e-6
45     if mask is None:
46         mask = np.where(img > 0)
47     bins = np.linspace(0, 255, num=L)
48     prob_dist, bins = np.histogram(img[mask].flatten(), bins=bins, density=
49                                     True)
50     # getting upper bin edges
51     gray_levels = bins[1:]
52     mu_t = np.sum(gray_levels*prob_dist)
53     max_var = 0
54     threshold = 0
55     for k in range(1, L-1):
56         mu_k = np.sum(gray_levels[:k]*prob_dist[:k])
57         w_k = np.sum(prob_dist[:k])
58         var_b = ((mu_t*w_k - mu_k)**2)/(w_k*(1-w_k)+epsilon) # OTSU
59         # threshold selection method: Eq. 18
60
61         if var_b > max_var:
62             max_var = var_b
63             threshold = gray_levels[k]

```

```

63     # print(f"threshold: {threshold}, max var_b: {max_var}")
64     return threshold
65
66 def compute_prob_foreground(img, threshold, flip=False, L=256):
67     """Given the image and threshold, computes the probability
68     of pixel values greater than threshold i.e. prob of foreground object.
69     """
70     bins = np.linspace(0, 255, num=L)
71     prob_dist, bins = np.histogram(img.flatten(), bins=bins, density=True)
72     gray_levels = bins[1:]
73     if flip:
74         return np.sum(prob_dist[np.where(gray_levels < threshold)])
75     else:
76         return np.sum(prob_dist[np.where(gray_levels > threshold)])
77
78
79 def otsu_single_channel(
80     img, L = 256, max_itr=10, flip=False):
81     """Performs multiple iterations of otsu algorithm, stops if
82     number of iterations is equal to the max_itr or probability of
83     foreground
84     object is less than or equal to the given estimate of probability.
85
86     Args:
87         img: input image (single channel)
88         L: number of gray levels, default=256.
89         max_itr: max number of iterations, default=5
90         flip: If yes, smaller pixels values are considered as foreground.
91     """
92     mask = None
93     for itr in range(max_itr):
94         threshold = otsu_single_itr(img, L=L, mask=mask)
95         if flip:
96             mask = np.where(img < threshold)
97         else:
98             mask = np.where(img > threshold)
99
100     print(f"Selected threshold: {threshold}")
101     segmented_img = np.zeros_like(img)
102     segmented_img[mask] = 255
103     return segmented_img.astype(np.uint8)
104
105
106 def get_segmentation_mask_using_RGB(img, flip=None, L=256, max_itr=None):
107     """computes segmentation mask using RGB channels"""
108     # mask using channel R
109     if flip is None:
110         flip = [False, False, False]
111     if max_itr is None:
112         max_itr = [1,1,1]
113     mask_R = otsu_single_channel(
114         img[...,0], flip=flip[0], L=L, max_itr=max_itr[0]
115     )
116
117     # mask using channel R
118     mask_G = otsu_single_channel(
119         img[...,1], flip=flip[1], L=L, max_itr=max_itr[1]
120     )

```

```

121
122 # mask using channel R
123 mask_B = otsu_single_channel(
124     img[...,2], flip=flip[2], L=L, max_itr=max_itr[2]
125 )
126
127 mask_combined = mask_R & mask_G & mask_B
128
129 return mask_combined, mask_R, mask_G, mask_B
130
131
132
133 def dilation(img, kernel_size=3):
134     """Dilation implemented for single channel images,
135     convert image to grayscale if it is 3 channel
136     """
137     if len(img.shape) == 3:
138         img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
139     height, width = img.shape
140     k = kernel_size//2
141     dilated_img = np.zeros_like(img)
142     for i in range(k, height - k):
143         for j in range(k, width - k):
144             dilated_img[i,j] = np.max(img[i-k:i+k+1, j-k:j+k+1])
145
146     return dilated_img
147
148 def erosion(img, kernel_size=3):
149     """Erosion implemented for single channel images,
150     convert image to grayscale if it is 3 channel
151     """
152     if len(img.shape) == 3:
153         img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
154     height, width = img.shape
155     k = kernel_size//2
156     eroded_img = np.zeros_like(img)
157     for i in range(k, height - k):
158         for j in range(k, width - k):
159             eroded_img[i,j] = np.min(img[i-k:i+k+1, j-k:j+k+1])
160     return eroded_img
161
162 def get_contour_mask(img):
163     """Expects a binary image at the input.
164     """
165     if len(img.shape) == 3:
166         img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
167     height, width = img.shape
168     k = 1
169     img = img.copy().astype(np.int32)
170     contour_mask = np.zeros_like(img)
171     for i in range(k, height - k):
172         for j in range(k, width - k):
173             if (img[i,j] > 0) and (np.sum(img[i-k:i+k+1, j-k:j+k+1]) < 9*img
174                 [i,j]):
175                 # if first condition is true, the sum of the pixel (i,j) and
176                 # its 8
177                 # neighbors can only be less than 9, when at least one of
178                 # its neighbors is 0

```



```

176         # which the condition we wanted to check for contour
177         # extraction.
178         contour_mask[i,j] = img[i,j]
179     return contour_mask.astype(np.uint8)
180
181 def get_zero_padded_image(img, k):
182     """zero pads input
183     """
184     height, width = img.shape[:2]
185     zero_padded_img = np.zeros((height+2*k, width+2*k))
186     zero_padded_img[k:k+height, k:k+width] = img
187     return zero_padded_img
188
189 def get_texture_map(img, kernel_size):
190     """Returns the texture map of the image, as the variance of
191     pixels in the NxN neighborhood.
192     """
193     if len(img.shape) == 3:
194         img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
195     height, width = img.shape
196     k = kernel_size//2
197     img = img.copy().astype(np.float32)
198     texture_map = np.zeros_like(img)
199     zero_padded_img = get_zero_padded_image(img, k)
200     for i in range(height):
201         for j in range(width):
202             local_var = np.var(zero_padded_img[i-k:i+k+1, j-k:j+k+1])
203             texture_map[i,j] = local_var
204     texture_map = np.where(np.isnan(texture_map), 0, texture_map)
205     texture_map = 255*texture_map/(np.max(texture_map) - np.min(texture_map)
206     )
207     return texture_map.astype(np.uint8)
208
209 def extract_contour(img, kernel_size, morphological_operator='grad'):
210     if morphological_operator == 'grad':
211         dilated = dilation(img, kernel_size)
212         erroded = erosion(img, kernel_size)
213         morph_out = dilated - erroded
214     elif morphological_operator == 'open':
215         erroded = erosion(img, kernel_size)
216         morph_out = dilation(erroded, kernel_size)
217
218     elif morphological_operator == 'close':
219         dilated = dilation(img, kernel_size)
220         morph_out = erosion(dilated, kernel_size)
221
222     contour_map = get_contour_mask(morph_out)
223     return contour_map
224
225
226
227 def closing(img, kernel_size):
228     output_img = img.copy()
229     output_img = dilation(output_img, kernel_size)
230     output_img = erosion(output_img, kernel_size)
231     return output_img
232

```

```

233 def opening(img, kernel_size):
234     output_img = img.copy()
235     output_img = erosion(output_img, kernel_size)
236     output_img = dilation(output_img, kernel_size)
237     return output_img
238
239 def get_segmentation_using_texture(img, kernel_sizes, L=256, flip=None,
    max_itr=None):
240
241     texture_maps = [get_texture_map(img, kernel_size=kernel_size) for
        kernel_size in kernel_sizes]
242     if flip is None:
243         flip = [False, False, False,]
244     if max_itr is None:
245         max_itr = [2,2,2]
246     masks = [otsu_single_channel(
247         texture_map, flip=flip[i], L=L, max_itr=max_itr[i]
248         ) for i, texture_map in enumerate(texture_maps)]
249
250     mask_combined = masks[0] & masks[1] & masks[2]
251     return mask_combined, masks[0], masks[1], masks[2]
252
253
254
255 if __name__ == '__main__':
256
257     # Task-1: flower_small
258     # segmentation mask using RGB
259     subdir = 'hw6'
260     image_name = 'flower_small'
261     # reading images...
262     img = read_image(subdir, f"{image_name}.jpg")
263     L = 256
264     max_itr = [2,2,2]
265     flip=[False, False, False]
266     mask_combined, mask_R, mask_G, mask_B = get_segmentation_mask_using_RGB(
267         img, flip=flip, L=L, max_itr=max_itr
268     )
269
270     save_image(mask_R, subdir, f'{image_name}-mask-R.jpg', True)
271     save_image(mask_G, subdir, f'{image_name}-mask-G.jpg', True)
272     save_image(mask_B, subdir, f'{image_name}-mask-B.jpg', True)
273     save_image(mask_combined, subdir, f'{image_name}-mask-combined.jpg',
        True)
274
275     morphological_kernel = 3
276     contour_map = extract_contour(mask_combined, morphological_kernel, 'grad
        ')
277     save_image(contour_map, subdir, f'{image_name}-contour-map.jpg', True)
278
279
280     texture_kernels = [9, 11, 13]
281     max_itr = [1,1,1]
282     flip=[False, False, False]
283     texture_mask_combined, mask_1, mask_2, mask_3 =
        get_segmentation_using_texture(
284         img, texture_kernels, flip=flip, L=L, max_itr=max_itr
285     )
286

```

```

287     save_image(mask_1, subdir, f'{image_name}-texture-mask-1.jpg', True)
288     save_image(mask_2, subdir, f'{image_name}-texture-mask-2.jpg', True)
289     save_image(mask_3, subdir, f'{image_name}-texture-mask-3.jpg', True)
290     save_image(texture_mask_combined, subdir, f'{image_name}-texture-mask-
        combined.jpg', True)
291
292     morphological_kernel = 3
293     contour_map = extract_contour(texture_mask_combined,
        morphological_kernel, 'close')
294     save_image(contour_map, subdir, f'{image_name}-texture-contour-map.jpg',
        True)
295
296     # Task-1: dog
297     # segmentation mask using RGB
298     subdir = 'hw6'
299     image_name = 'dog_small'
300     # reading images...
301     img = read_image(subdir, f"{image_name}.jpg")
302     L = 256
303     max_itr = [2, 2, 2]
304     flip=[True, True, True]
305     mask_combined, mask_R, mask_G, mask_B = get_segmentation_mask_using_RGB(
306         img, flip=flip, L=L, max_itr=max_itr
307     )
308
309     save_image(mask_R, subdir, f'{image_name}-mask-R.jpg', True)
310     save_image(mask_G, subdir, f'{image_name}-mask-G.jpg', True)
311     save_image(mask_B, subdir, f'{image_name}-mask-B.jpg', True)
312     save_image(mask_combined, subdir, f'{image_name}-mask-combined.jpg',
        True)
313
314     morphological_kernel = 3
315     contour_map = extract_contour(mask_combined, morphological_kernel, 'open
        ')
316     save_image(contour_map, subdir, f'{image_name}-contour-map.jpg', True)
317
318
319     texture_kernels = [5,7, 11]
320     max_itr = [3, 2, 2]
321     flip=[True, True, True]
322     texture_mask_combined, mask_1, mask_2, mask_3 =
        get_segmentation_using_texture(
323         img, texture_kernels, flip=flip, L=L, max_itr=max_itr
324     )
325
326     save_image(mask_1, subdir, f'{image_name}-texture-mask-1.jpg', True)
327     save_image(mask_2, subdir, f'{image_name}-texture-mask-2.jpg', True)
328     save_image(mask_3, subdir, f'{image_name}-texture-mask-3.jpg', True)
329     save_image(texture_mask_combined, subdir, f'{image_name}-texture-mask-
        combined.jpg', True)
330
331     morphological_kernel = 5
332     contour_map = extract_contour(texture_mask_combined,
        morphological_kernel, 'open')
333     save_image(contour_map, subdir, f'{image_name}-texture-contour-map.jpg',
        True)
334
335     # Task-2: Fire-hydrant
336     subdir = 'hw6'

```

```
337 image_name = 'fire_hydrant'
338 print(f"Running for figure: {image_name}")
339 # reading images...
340 img = read_image(subdir, f"{image_name}.jpg")
341 foreground_prob = 0.15
342 L = 256
343 max_itr = [2,2,1]
344 flip=[False, False, False]
345 mask_combined, mask_R, mask_G, mask_B = get_segmentation_mask_using_RGB(
346     img, flip=flip, L=L, max_itr=max_itr
347 )
348
349 save_image(mask_R, subdir, f'{image_name}-mask-R.jpg', True)
350 save_image(mask_G, subdir, f'{image_name}-mask-G.jpg', True)
351 save_image(mask_B, subdir, f'{image_name}-mask-B.jpg', True)
352 save_image(mask_combined, subdir, f'{image_name}-mask-combined.jpg',
353     True)
354
355 morphological_kernel = 3
356 contour_map = extract_contour(mask_combined, morphological_kernel, 'open')
357
358 save_image(contour_map, subdir, f'{image_name}-contour-map.jpg', True)
359
360 texture_kernels = [3,5,7]
361 max_itr = [1, 1, 1]
362 flip=[False, False, False]
363 texture_mask_combined, mask_1, mask_2, mask_3 =
364     get_segmentation_using_texture(
365         img, texture_kernels, flip=flip, L=L, max_itr=max_itr
366     )
367
368 save_image(mask_1, subdir, f'{image_name}-texture-mask-1.jpg', True)
369 save_image(mask_2, subdir, f'{image_name}-texture-mask-2.jpg', True)
370 save_image(mask_3, subdir, f'{image_name}-texture-mask-3.jpg', True)
371 save_image(texture_mask_combined, subdir, f'{image_name}-texture-mask-
372     combined.jpg', True)
373
374 morphological_kernel = 3
375 contour_map = extract_contour(texture_mask_combined,
376     morphological_kernel, 'grad')
377 save_image(contour_map, subdir, f'{image_name}-texture-contour-map.jpg',
378     True)
379
380 # Task-2: flowers
381 subdir = 'hw6'
382 image_name = 'flowers'
383 print(f"Running for figure: {image_name}")
384 # reading images...
385 img = read_image(subdir, f"{image_name}.jpg")
386 foreground_prob = 0.001
387 L = 256
388 max_itr = [1,1,1]
389 flip=[False, True, False]
390 mask_combined, mask_R, mask_G, mask_B = get_segmentation_mask_using_RGB(
391     img, flip=flip, L=L, max_itr=max_itr
392 )
```

```
390     save_image(mask_R, subdir, f'{image_name}-mask-R.jpg', True)
391     save_image(mask_G, subdir, f'{image_name}-mask-G.jpg', True)
392     save_image(mask_B, subdir, f'{image_name}-mask-B.jpg', True)
393     save_image(mask_combined, subdir, f'{image_name}-mask-combined.jpg',
394               True)
395
396     morphological_kernel = 3
397     contour_map = extract_contour(mask_combined, morphological_kernel, '
398         close')
399     save_image(contour_map, subdir, f'{image_name}-contour-map.jpg', True)
400
401     texture_kernels = [3,5,7]
402     max_itr = [1, 1, 1]
403     flip=[False, False, False]
404     texture_mask_combined, mask_1, mask_2, mask_3 =
405         get_segmentation_using_texture(
406             img, texture_kernels, flip=flip, L=L, max_itr=max_itr
407         )
408
409     save_image(mask_1, subdir, f'{image_name}-texture-mask-1.jpg', True)
410     save_image(mask_2, subdir, f'{image_name}-texture-mask-2.jpg', True)
411     save_image(mask_3, subdir, f'{image_name}-texture-mask-3.jpg', True)
412     save_image(texture_mask_combined, subdir, f'{image_name}-texture-mask-
413         combined.jpg', True)
414
415     morphological_kernel = 3
416     contour_map = extract_contour(texture_mask_combined,
417         morphological_kernel, 'close')
418     save_image(contour_map, subdir, f'{image_name}-texture-contour-map.jpg',
419         True)
```

Listing 1: Example Python code